

Chapter 1: Implementation of link list

1.What is data structure?

Data Structure can be defined as the group of data elements which provides an efficient way of storing and organising data in the computer so that it can be used efficiently. Some examples of Data Structures are arrays, Linked List, Stack, Queue, etc.

- *A storage that is used to store and organize data.*
- *Arrangement of data on the computer, so that data can be accessed efficiently and easily.*

a)Types of Data Structures

There are two types of data structures:

- Primitive data structure
- Non-primitive data structure

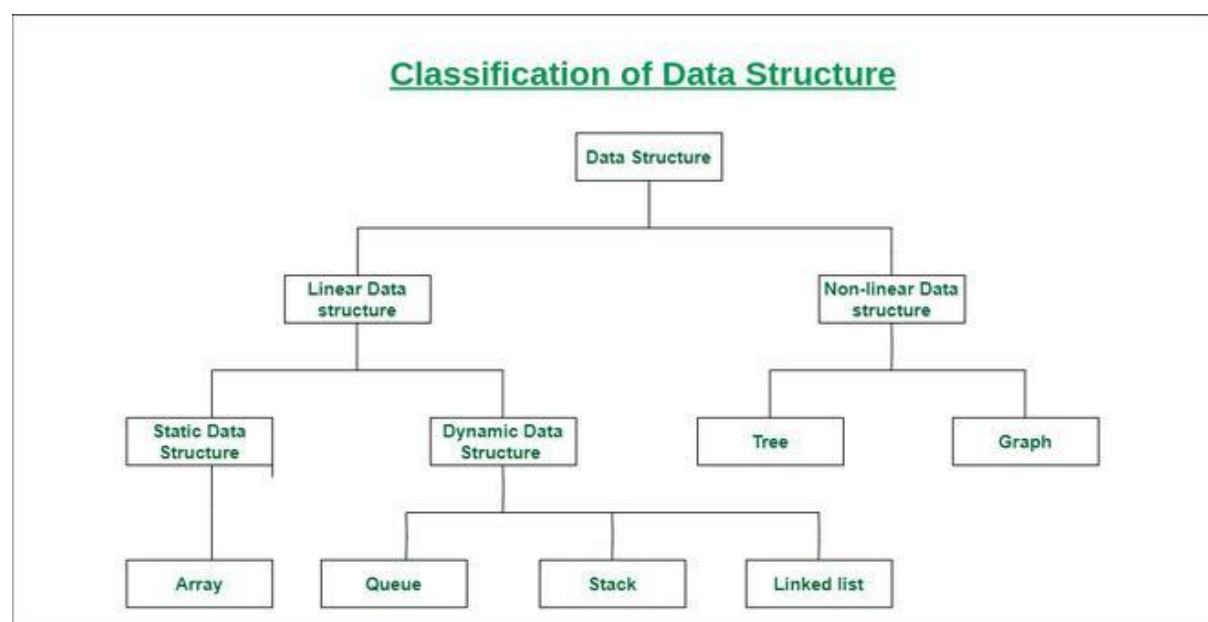
a.1)Primitive Data structure

The primitive data structures are primitive data types. The int, char, float, double, and pointer are the primitive data structures that can hold a single value.

a.2)Non-Primitive Data structure

The non-primitive data structure is divided into two types:

- Linear data structure
- Non-linear data structure



b)Need Of Data structure:

- Data structure modification is easy.
- It requires less time.
- Save storage memory space.
- Data representation is easy.
- Easy access to the large database.

c)Major Operations

- **Searching:** We can search for any element in a data structure.
- **Sorting:** We can sort the elements of a data structure either in an ascending or descending order.
- **Insertion:** We can also insert the new element in a data structure.
- **Updation:** We can also update the element, i.e., we can replace the element with another element.
- **Deletion:** We can also perform the delete operation to remove the element from the data structure.

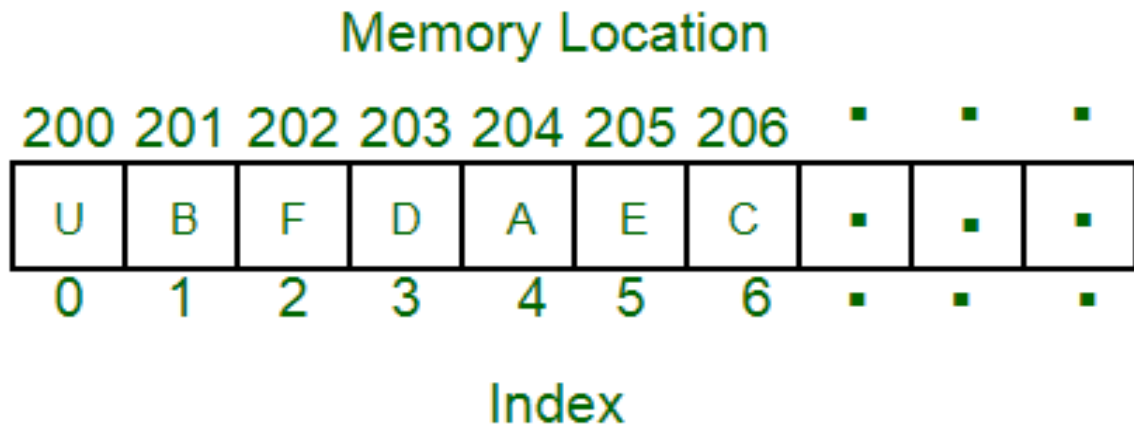
Array in Data Structure

Arrays are defined as the collection of similar types of data items stored at contiguous memory locations. It is one of the simplest data structures where each data element can be randomly accessed by using its index number.

Properties of array

There are some of the properties of an array that are listed as follows -

- Each element in an array is of the same data type and carries the same size that is 4 bytes.
- Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.



Basic operations

Now, let's discuss the basic operations supported in the array -

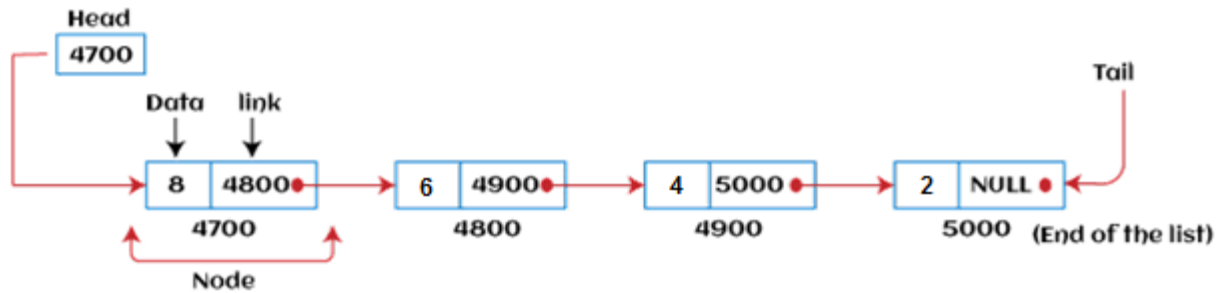
- Traversal - This operation is used to print the elements of the array.
- Insertion - It is used to add an element at a particular index.
- Deletion - It is used to delete an element from a particular index.
- Search - It is used to search an element using the given index or by the value.
- Update - It updates an element at a particular index.

Linked list

Linked list is a linear data structure that includes a series of connected nodes. Linked list can be defined as the nodes that are randomly stored in the memory. A node in the linked list contains two parts, i.e., first is the data part and second is the address part. The last node of the list contains a pointer to the null. After array, linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

Representation of a Linked list

Linked list can be represented as the connection of nodes in which each node points to the next node of the list. The representation of the linked list is shown below -



Why use linked list over array?

Linked list is a data structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays -

- The size of the array must be known in advance before using it in the program.
- Increasing the size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.
- All the elements in the array need to be contiguously stored in the memory. Inserting an element in the array needs shifting of all its predecessors.

Linked list is useful because -

- It allocates the memory dynamically. All the nodes of the linked list are non-contiguously stored in the memory and linked together with the help of pointers.
- In linked list, size is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

Types of Linked list

- **Singly-linked list** - Singly linked list can be defined as the collection of an ordered set of elements. A node in the singly linked list consists of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node, while the link part of the node stores the address of its immediate successor.
- **Doubly linked list** - Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly-linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), and pointer to the previous node (previous pointer).
- **Circular singly linked list** - In a circular singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.

- **Circular doubly linked list** - Circular doubly linked list is a more complex type of data structure in which a node contains pointers to its previous node as well as the next node. Circular doubly linked list doesn't contain NULL in any of the nodes. The last node of the list contains the address of the first node of the list. The first node of the list also contains the address of the last node in its previous pointer.

Q2.What is Singly Link List Implementation?

Singly linked list can be defined as the collection of ordered set of elements. The number of elements may vary according to need of the program. A node in the singly linked list consist of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.

One way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only next pointer, therefore we can not traverse the list in the reverse direction.

Consider an example where the marks obtained by the student in three subjects are stored in a linked list as shown in the figure.



Various Operations of Singly Link List:

- a)Insert from head
- b)Insert from tail
- c)Delete from head
- d)Delete from tail
- c)Display

Program Implementation:

```
class Node {  
    constructor(data) {  
        this.value = data;  
        this.next = null;  
    }  
}
```

```

class Linkedlist {
  constructor(data) {
    //create a node
    const newnode = new Node(data);
    //initialize head and tail
    this.head = newnode;
    this.tail = newnode;
    //initialize lenth
    this.length = 1;
  }

  push(value) {
    const newNode = new Node(value);
    // adding a node to an empty linkedlist
    if (!this.length) {
      this.head = newNode;
      this.tail = this.head;
    } else {
      this.tail.next = newNode;
      this.tail = newNode;
    }
    this.length++;
    return this;
  }

  pop() {
    if (!this.length) return undefined;
    // one item in list
    if (this.length === 1) {
      let temp = this.head;
      this.head = null;
      this.tail = null;
      this.length--;
      return temp;
    } else {
      let temp = this.head;
      let prev = this.head;
      while (temp.next) {
        prev = temp;
        temp = temp.next;
      }
      this.tail = prev;
      this.tail.next = null;
      this.length--;
      return temp;
    }
  }

  unshift(value) {

```

```

const newNode = new Node(value);
if (!this.length) {
  this.head = newNode;
  this.tail = this.head;
} else {
  newNode.next = this.head;
  this.head = newNode;
}
this.length++;
return this;
}

shift() {
  if (!this.length) return undefined;
  let temp = this.head;
  if (this.length === 1) {
    this.head = null; // this will break pointer which head is pointing
    this.tail = null;
  } else {
    this.head = this.head.next;
    temp.next = null;
  }
  this.length--;
  return temp;
}
}

const ll = new Linkedlist(2);
console.log(ll);
ll.push(3);
ll.push(4);
ll.push(5);
ll.push(6);
ll.push(7);
ll.push(8);
ll.unshift(10);
console.log(JSON.stringify(ll));
let p = ll.pop();
let p1 = ll.shift();
console.log(JSON.stringify(ll));
console.log(p);
console.log(p1);

```

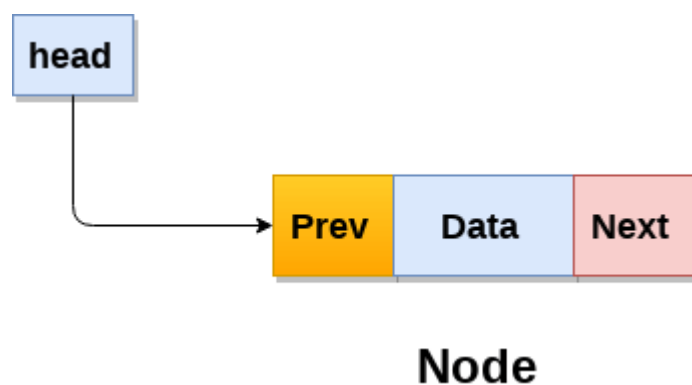
Applications of linked list in the real world:

1. Image viewer – Previous and next images are linked and can be accessed by the next and previous buttons.
2. Previous and next page in a web browser – We can access the previous and next URL searched in a web browser by pressing the back and next buttons since they are linked as a linked list.

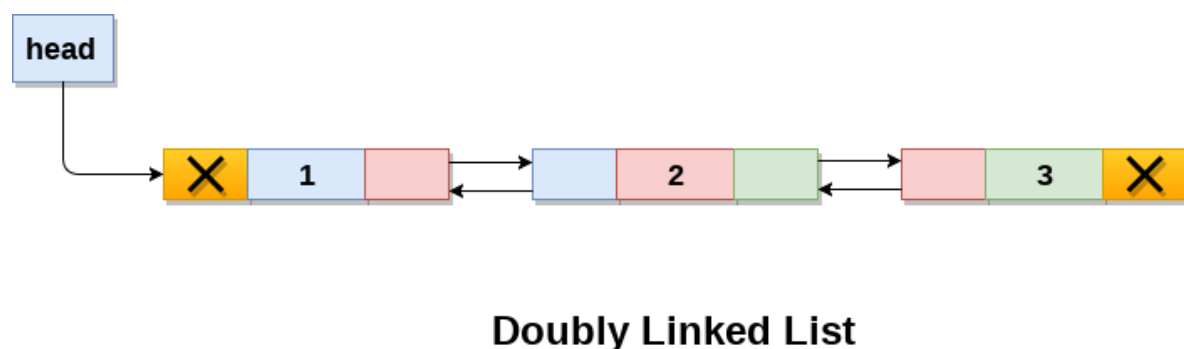
3. Music Player – Songs in the music player are linked to the previous and next songs. So you can play songs either from starting or ending of the list.

Q3.What is Doubly linked list?

Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.



A doubly linked list containing three nodes having numbers from 1 to 3 in their data part, is shown in the following image.



Operations of Doubly Link List:

- a)Insert from head
- b)Insert from tail
- c)Delete from head
- d)Delete from tail
- e)Displa

Program Implementation of Doubly link list:

```
class node {
  constructor(value) {
    this.value = value;
    this.pre = null;
    this.next = null;
  }
}
class Doublylinkedlist {
  constructor(value) {
    const newNode = new node(value);
    //initialize head and tail
    this.head = newNode;
    this.tail = newNode;
    this.length = 1;
  }

  push(value) {
    const newNode = new node(value);
    // adding a node to an empty linkedlist
    if (!this.length) {
      this.head = newNode;
      this.tail = newNode;
    } else {
      this.tail.next = newNode;
      newNode.pre = this.tail;
      this.tail = newNode;
    }
    this.length++;
    return this;
  }

  pop() {
    //if list is empty
    if (!this.length) return undefined;
    let temp = this.tail; //as we pop from tail
    if (this.length == 1) {
      this.head = null;
      this.tail = null;
    } else {
      temp.pre.next = null; //break the pointer
      this.tail = temp.pre; //update position of tail
      temp.pre = null;
    }
    this.length--;
    return temp;
    //one item is there and tail points to null

    //n items update tail and update length
  }
}
```

```

}
unshift(value) {
  const newNode = new Node(value);
  if (!this.length) {
    this.head = newNode;
    this.tail = newNode;
  } else {
    newNode.next = this.head;
    this.head.prev = newNode;
    this.head = newNode;
  }
  this.length++;
  return this;
}

shift() {
  if (!this.length) return undefined;
  let temp = this.head;
  if (this.length === 1) {
    this.head = null;
    this.tail = null;
  } else {
    this.head = temp.next;
    this.head.prev = null;
  }
  temp.next = null;
  temp.prev = null;
  this.length--;
  return temp;
}
}

const dl = new Doublylinkedlist(10);
console.log(dl);
dl.push(20);
dl.push(5);
dl.push(6);
dl.push(7);
dl.push(8);
console.log(JSON.stringify(dl, getCircularReplacer()));
//console.log(JSON.stringify(dl));

```

Application of Doubly Linked Lists:

1. Redo and undo functionality.
2. Use of the Back and forward button in a browser.
3. The most recently used section is represented by the Doubly Linked list.

4. Other Data structures like Stack, HashTable, and BinaryTree can also be applied by Doubly Linked List.

Advantages Of Linked List:

- **Dynamic data structure:** A linked list is a dynamic arrangement so it can grow and shrink at runtime by allocating and deallocating memory. So there is no need to give the initial size of the linked list.
- **No memory wastage:** In the Linked list, efficient memory utilization can be achieved since the size of the linked list increase or decrease at run time so there is no memory wastage and there is no need to pre-allocate the memory.
- **Implementation:** Linear data structures like stacks and queues are often easily implemented using a linked list.
- **Insertion and Deletion Operations:** Insertion and deletion operations are quite easier in the linked list. There is no need to shift elements after the insertion or deletion of an element only the address present in the next pointer needs to be updated.

Disadvantages Of Linked List:

- **Memory usage:** More memory is required in the linked list as compared to an array. Because in a linked list, a pointer is also required to store the address of the next element and it requires extra memory for itself.
- **Traversal:** In a Linked list traversal is more time-consuming as compared to an array. Direct access to an element is not possible in a linked list as in an array by index. For example, for accessing a node at position n, one has to traverse all the nodes before it.
- **Reverse Traversing:** In a singly linked list reverse traversing is not possible, but in the case of a doubly-linked list, it can be possible as it contains a pointer to the previously connected nodes with each node. For performing this extra memory is required for the back pointer hence, there is a wastage of memory.
- **Random Access:** Random access is not possible in a linked list due to its dynamic memory allocation.